UNIFAAT 2026.2

02 - CSS, Seletores e JS Reativo

Prof. Luan Lourenço

O repositório do bimestre

`github.com/luan-tavares/unifaat-frontend-bimestre-01`

Mesmo repositório da aula 01 — o projeto continua evoluindo nele.

Revisão rápida

```
# o que já sabemos
• DOM = árvore de nós gerada
pelo parsing de HTML
• JS acessa essa árvore pela Vendor API
• append / prepend inserem nós
```

```
# o que vem agora
• querySelector e querySelectorAll
com profundidade
• CSS como segundo consumidor
dos mesmos seletores
• JS orientado a eventos
```

O traço distintivo do frontend é o JS reativo.

`querySelector` vs. `querySelectorAll`

console.js

```
// o primeiro que casa → Element | null
const el = document.querySelector(".destaque");
// todos que casam → NodeList (não é array puro)
const lista = document.querySelectorAll(".destaque");
```

A sintaxe dos seletores é a mesma do CSS: a mesma linguagem, dois contextos.

Cardinalidade: id vs. class

```
# #id — 1 para 1
um elemento, um id único
no documento

querySelector("#cabecalho")
→ sempre 1 elemento
```

```
# .class — n para n
um elemento pode ter várias classes;
uma classe pode estar em muitos
elementos

querySelectorAll(".card")
→ NodeList com N elementos
```

É a cardinalidade que explica por que id devolve um e class devolve uma coleção.

`HTMLElement`, `NodeList` e a hierarquia

```
# cadeia de herança
HTMLElement herda de
Element herda de
Node
```

```
# o que cada um traz
Node: base — elemento, texto, comentário
Element: nós de tags HTML e SVG
HTMLElement: classList, style, innerHTML
```

`NodeList` tem índice e `length`, mas **não é um array**: alguns métodos de array não existem nela.

Navegando na árvore

console.js

```
destaque.children           // filhos diretos → HTMLCollection
destaque.firstElementChild  // primeiro filho   → Element | null
destaque.lastElementChild   // último filho      → Element | null
destaque.parentElement      // elemento pai      → Element | null
destaque.closest(".box")    // sobe até um ancestral → Element | null
```

`closest()` brilha dentro de handlers: sobe a partir do `event.target` até o elemento clicável.

`style`: CSS via JS

```
# regra de conversão
CSS: kebab-case
background-color
font-size

JS: camelCase
backgroundColor
fontSize
```

console.js

```js
const el = document.querySelector(".destaque");
// equivale a background-color
el.style.backgroundColor = "#f0f0f0";
// equivale a font-size
el.style.fontSize = "18px";
```

element.classList

console.js

```
// <div id="card" class="box active highlight">
const el = document.querySelector("#card");
el.classList[0]            // "box"
el.classList.length       // 3
el.classList.add("novo")        // adiciona
el.classList.remove("active")   // remove
el.classList.toggle("aberto")  // alterna → boolean
el.classList.contains("box")   // existe? → boolean
el.classList.replace("box", "caixa")
```

! Um `DOMTokenList` parece array, mas não é: sem `map`/`filter`. Converta com `Array.from()`.

CSS: a mesma linguagem de seleção

basicos.css

```
tag   { }  /* por tag */
.cls  { }  /* por classe */
#id   { }  /* por id */
A B   { }  /* descendente */
A > B { }  /* filho direto */
A + B { }  /* irmão imediato */
A ~ B { }  /* irmãos seguintes */
```

atributo-e-estado.css

```
[disabled]       { }
[type="text"]    { }
[href^="https"]  { }
[src$=".png"]    { }
[class*="alert"] { }
:first-child { }  :last-child { }
:nth-child(n) { }
:hover { }  :focus { }
```

Todos funcionam também dentro de `querySelector` e `querySelectorAll`.

`display`: o comportamento no fluxo

```
# block
ocupa 100% da largura,
quebra linha
ex.: div, p
```

```
# inline
só o espaço do conteúdo,
sem width/height
ex.: span, a
```

```
# inline-block
não quebra linha, mas aceita
dimensões e margens
```

```
# flex
container com controle avançado
de alinhamento dos filhos
```

Responsividade e media queries

responsivo.css

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
/* aplica quando a tela tem até 768px */
@media (max-width: 768px) {
  body { background: lightblue; }
}
```

Mobile first: escreva pra tela pequena primeiro e adicione breakpoints com `min-width`.

JS reativo: o traço do frontend

- **Evento:** ação do usuário ou do navegador — `click`, `input`, `submit`...
- **Listener:** função callback chamada automaticamente quando o evento ocorre
- **Propagação:** o evento sobe pela árvore (bubbling); `stopPropagation()` interrompe

---

Não é só manipular o DOM — é reagir ao que o usuário faz.

`addEventListener` na prática

console.js

```
container.addEventListener("click", function (event) {
// onde o clique aconteceu (pode ser um filho)
  console.log("target:", event.target);
// onde o listener foi registrado
  console.log("currentTarget:", event.currentTarget);
// sobe até o elemento clicável
  const item = event.target.closest(".item");
// impede propagação pros pais
  event.stopPropagation();
});
```

`type="module"`: vários arquivos

Scripts de frontend crescem — separar em arquivos é inevitável.

index.html

```
<script type="module" src="./app.js"></script>
```

- → Habilita `import` / `export` nativos no navegador
- → Cada arquivo tem escopo próprio — sem poluir variáveis globais

---

Sem aprofundar em módulos agora: só justificar por que um script inline único não basta.

CSS como commodity

Escrever CSS puro do zero é cada vez mais raro em produção: Bootstrap e Tailwind entregam classes prontas pra layout, espaçamento e componente.

```
# css manual
essencial pra entender seletor,
display, cascata e especificidade
```

```
# frameworks
resolvem a estilização repetitiva
em produção
```

Quem entende a base usa framework com consciência, não como caixa-preta.

Referência rápida: API do DOM

console.js

```
querySelector(sel)          → Element | null
querySelectorAll(sel)       → NodeList
classList.add(cls)          → undefined
classList.remove(cls)       → undefined
classList.toggle(cls)       → boolean
classList.contains(cls)     → boolean
element.remove()           → undefined
parentElement               → Element | null
closest(sel)                → Element | null
children                   → HTMLCollection
first/lastElementChild     → Element | null
```

Seletores, CSS e eventos.

A Vendor API ganhou profundidade e o CSS entrou como segundo consumidor dos mesmos seletores. A seguir: **Vite, formulários e listeners**.

tarefa

TF 02: CSS, Seletores e JS Reativo

Entrega pelo formulário abaixo —
docs.google.com/forms/d/e/1FAIpQLSdJ1DIp8VmDNyzQV2haY4HWodQPi2OSSbwb_CHGk5LzfYtHBw/viewform